

บทที่ 10 การตรวจจับค่าผิดปกติ (Outlier Detection)

วัตถุประสงค์การเรียนรู้

1. อธิบายความหมาย หลักการ และประเภทของ Outlier ได้แก่ Global, Contextual และ Collective Outlier รวมถึงแนวทางการตรวจจับ 4 แบบ ได้แก่ Statistical-Based, Distance-Based, Density-Based และ Model-Based Approaches
2. ใช้วิธี Z-score เพื่อตรวจจับ Outlier โดยอาศัยการกระจายแบบ Normal Distribution กำหนด Threshold ที่เหมาะสม และเข้าใจข้อจำกัดของวิธีนี้เมื่อข้อมูลไม่เป็น Gaussian
3. ประยุกต์ใช้วิธี IQR (Interquartile Range) เพื่อระบุจุดข้อมูลที่อยู่นอกช่วง $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ และเปรียบเทียบความทนทานต่อ Outlier กับวิธี Z-score
4. ใช้ One-Class SVM สำหรับการตรวจจับ Outlier แบบ Novelty Detection โดยหา Hyperplane ที่แยกข้อมูลปกติออกจาก Origin ด้วย Margin สูงสุด และปรับพารามิเตอร์ ν เพื่อควบคุมสัดส่วน Outlier
5. ใช้ Isolation Forest เพื่อแยกแยะ Outlier โดยอาศัยการสุ่ม Feature และ Split Value โดยเข้าใจว่า Outlier ถูกแยกได้ด้วยความลึกเฉลี่ยน้อยกว่า และปรับพารามิเตอร์ contamination ให้เหมาะสม
6. ใช้ DBSCAN สำหรับการตรวจจับ Outlier ในฐานะจุดข้อมูลที่ไม่อยู่ใน Cluster ใดๆ โดยปรับพารามิเตอร์ ϵ และ $\min_samples$ และเข้าใจความเหมาะสมกับข้อมูลที่ Cluster มีรูปทรงซับซ้อน

7. ใช้ Local Outlier Factor (LOF) เพื่อตรวจจับ Outlier ตามความหนาแน่นเฉพาะถิ่น โดยคำนวณ Local Reachability Density และเปรียบเทียบกับเพื่อนบ้าน k คนแรก และเข้าใจข้อดีเหนือวิธี Global เช่น Z-score ในข้อมูลที่มีความหนาแน่นไม่สม่ำเสมอ
8. เปรียบเทียบประสิทธิภาพของวิธีการตรวจจับ Outlier ทั้ง 6 แบบ ได้แก่ Z-score, IQR, One-Class SVM, Isolation Forest, DBSCAN และ LOF บนชุดข้อมูล Credit Card Fraud Detection จริง โดยวิเคราะห์ Recall, False Positive และเลือกวิธีที่เหมาะสมกับลักษณะข้อมูลและเป้าหมายการวิเคราะห์

10.1 บทนำ

การตรวจจับค่าผิดปกติ (Outlier Detection) หรือที่รู้จักในชื่อ Anomaly Detection เป็นเทคนิคที่ใช้ระบุการสังเกตในชุดข้อมูลที่เบี่ยงเบนอย่างมีนัยสำคัญจากข้อมูลส่วนใหญ่ การสังเกตเหล่านี้มักเรียกว่า Outlier หรือ Anomaly การตรวจจับ Outlier มีประโยชน์ในหลายสาขา เช่น การเงิน สาธารณสุข และความปลอดภัยทางไซเบอร์ สำหรับการระบุรูปแบบที่ผิดปกติ การทุจริต หรือข้อผิดพลาดในข้อมูล

วิธีการตรวจจับ Outlier ที่นิยมใช้ใน Scikit-learn มีดังนี้:

1. Isolation Forest: วิธีการที่แยกการสังเกตโดยการสุ่มเลือก Feature แล้วสุ่มเลือกค่า Split ระหว่างค่าสูงสุดและต่ำสุดของ Feature ที่เลือก
2. Local Outlier Factor (LOF): วิธีการที่คำนวณความหนาแน่นเฉพาะถิ่นของจุดข้อมูลและเปรียบเทียบกับความหนาแน่นของเพื่อนบ้าน จุดข้อมูลที่มีความหนาแน่นเฉพาะถิ่นต่ำถือเป็น Outlier
3. One-Class SVM: วิธีการที่เรียนรู้ขอบเขตที่แยกข้อมูลส่วนใหญ่ออกจากข้อมูล Outlier
4. DBSCAN: อัลกอริทึม Clustering ตามความหนาแน่นที่สามารถใช้ตรวจจับ Outlier โดยจัดกลุ่มจุดข้อมูลที่อยู่ใกล้กันตาม Threshold ความหนาแน่น (Eps)

และจำนวนจุดขั้นต่ำ (MinPts) จุดข้อมูลที่ไม่ได้อยู่ในกลุ่มหนาแน่นใดๆ ถือเป็น Outlier

5. IQR: การวัดทางสถิติที่ใช้ตรวจจับ Outlier นิยามเป็นผลต่างระหว่าง Quartile ที่สาม (Q3) และ Quartile แรก (Q1) ซึ่งเป็นการวัดที่ทนทานต่อ Outlier

10.2 การตรวจจับ Outlier

การตรวจจับ Outlier เป็นงานสำคัญในการวิเคราะห์ข้อมูล มุ่งระบุจุดข้อมูลที่เบี่ยงเบนอย่างมีนัยสำคัญจากบรรทัดฐาน ส่วนนี้แนะนำวิธีการตรวจจับ Outlier ต่างๆ โดยใช้แพ็คเกจ Scikit-learn

Outlier คือจุดข้อมูลที่แสดงพฤติกรรมผิดปกติเมื่อเทียบกับข้อมูลส่วนใหญ่ในชุดข้อมูล การตรวจจับ Outlier มีความสำคัญในหลายสาขา รวมถึงการตรวจจับการทุจริต การควบคุมคุณภาพ และการตรวจจับความผิดปกติ Scikit-learn มีเครื่องมือหลากหลายสำหรับนำวิธีการตรวจจับ Outlier ไปใช้ได้อย่างมีประสิทธิภาพ

10.2.1 ทฤษฎีการตรวจจับค่าผิดปกติ (Outlier Detection)

Mahalanobis Distance

$$D_M(x) = \sqrt{(x - \mu)^T \Sigma^{-1} (x - \mu)}$$

โดยที่:

$DM(x)$ = Mahalanobis Distance ของจุดข้อมูล x จากศูนย์กลางการแจกแจง

μ = เวกเตอร์ค่าเฉลี่ย (Mean Vector)

Σ^{-1} = เมทริกซ์ผกผันของ Covariance Matrix

ข้อดี: Mahalanobis Distance คำนึงถึงความสัมพันธ์ระหว่างตัวแปร (Correlation)

และความแปรปรวนแต่ละมิติ ทำให้เปรียบเทียบข้อมูลในหลายมิติได้อย่างยุติธรรม

ต่างจาก Euclidean Distance ที่ไม่คำนึงถึงความสัมพันธ์

Local Outlier Factor (LOF)

$$LOF_k(p) = \frac{\sum_{o \in N_k(p)} lrd_k(o)}{\frac{|N_k(p)|}{lrd_k(p)}}$$

โดยที่:

$LOF_k(p)$ = Local Outlier Factor ของจุดข้อมูล p โดยใช้ k เพื่อนบ้าน

$N_k(p)$ = เซตเพื่อนบ้าน k อันดับแรกของ p

$lrd_k(p)$ = Local Reachability Density ของ p

$lrd_k(o)$ = Local Reachability Density ของเพื่อนบ้าน o

การตีความ: $LOF \approx 1$ จุดปกติที่อยู่ในกลุ่ม, $LOF > 1$ อาจเป็นค่าผิดปกติ (ยิ่งสูงยิ่งผิดปกติ)

LOF เหมาะกับข้อมูลที่มีความหนาแน่นต่างกันในแต่ละบริเวณ ต่างจาก Global Outlier Detection ที่ใช้เกณฑ์เดียวกันทั้งหมด

Grubbs' Test Statistic

$$G = \frac{\max |x_i - \bar{x}|}{s}$$

โดยที่:

G = สถิติทดสอบ Grubbs

x_i = ค่าข้อมูลที่สงสัยว่าเป็นค่าผิดปกติ

$\bar{x}$ = ค่าเฉลี่ยของข้อมูลทั้งชุด

s = ส่วนเบี่ยงเบนมาตรฐานของข้อมูล

หลักการ: ค่า G ที่คำนวณได้จะนำไปเปรียบเทียบกับค่าวิกฤต (Critical Value)

จากตาราง Grubbs หากค่า $G >$ ค่าวิกฤต

แสดงว่าค่านั้นเป็นค่าผิดปกติอย่างมีนัยสำคัญทางสถิติ

ข้อจำกัด: Grubbs Test ใช้ได้กับข้อมูลที่มีการแจกแจงปกติเท่านั้น
และทดสอบได้ทีละหนึ่งค่า

การตรวจจับ Outlier หรือ Anomaly Detection เป็นงาน Data Mining ที่มุ่งระบุจุดข้อมูลที่เบี่ยงเบนอย่างมีนัยสำคัญจากรูปแบบปกติในชุดข้อมูล Tan et al. (2006) อธิบายว่า Outlier คือจุดข้อมูลที่มีลักษณะแตกต่างจากข้อมูลส่วนใหญ่ในลักษณะที่ทำให้ต้องสงสัยว่าเกิดจากกลไกที่แตกต่างกัน สาเหตุของ Outlier อาจเกิดจากข้อผิดพลาดในการวัด ข้อผิดพลาดในการบันทึกข้อมูล หรือเป็นเหตุการณ์ที่แท้จริงที่หาได้ยาก เช่น การทุจริตทางการเงิน การโจมตีทางไซเบอร์ หรือข้อบกพร่องในกระบวนการผลิต

ประเภทของ Outlier

Tan et al. (2006) จำแนก Outlier ออกเป็น 3 ประเภทหลัก ได้แก่ (1) Global Outlier (Point Anomaly) คือจุดข้อมูลที่มีค่าเบี่ยงเบนอย่างมากจากข้อมูลทั้งหมด เช่น คนที่มีรายได้สูงผิดปกติในฐานะข้อมูลลูกค้า (2) Contextual Outlier (Conditional Anomaly) คือจุดข้อมูลที่ถูกเป็น Outlier ในบริบทหนึ่ง แต่ไม่ใช่ในบริบทอื่น เช่น อุณหภูมิ 35 องศาเซลเซียสในฤดูหนาวเป็นสิ่งผิดปกติ แต่ในฤดูร้อนเป็นเรื่องปกติ (3) Collective Outlier คือกลุ่มของจุดข้อมูลที่เกี่ยวข้องกันเมื่อพิจารณาาร่วมกันถือเป็น Outlier แม้ว่าแต่ละจุดอาจดูปกติเมื่อพิจารณาเดี่ยวๆ

แนวทางการตรวจจับ Outlier

Tan et al. (2006) สรุปแนวทางหลักในการตรวจจับ Outlier ดังนี้: (1) Statistical-Based Approaches ใช้การกระจายทางสถิติของข้อมูลเป็นฐาน เช่น การสมมติว่าข้อมูลปกติปฏิบัติตาม Gaussian Distribution จุดที่อยู่ห่างจากค่าเฉลี่ยมากกว่า k เท่าของส่วนเบี่ยงเบนมาตรฐานถือเป็น Outlier (2) Distance-Based Approaches จุดข้อมูลที่มีเพื่อนบ้านน้อย (ระยะห่างจากเพื่อนบ้านสูง) ถือเป็น Outlier (3) Density-Based Approaches จุดที่มีความหนาแน่นเฉพาะถิ่นต่ำกว่าเพื่อนบ้านอย่างมีนัยสำคัญถือเป็น Outlier (4)

Model-Based Approaches สร้างโมเดลที่แทนข้อมูลปกติ จุดที่ไม่ตรงกับโมเดลเป็น Outlier

Z-score: วิธีการทางสถิติพื้นฐาน

Z-score เป็นวิธีการตรวจจับ Outlier ที่ง่ายและตีความได้ง่าย Hastie, Tibshirani และ Friedman (2009) อธิบายว่า Z-score

วัดจำนวนส่วนเบี่ยงเบนมาตรฐานที่จุดข้อมูลอยู่ห่างจากค่าเฉลี่ย: $Z = \frac{x - \mu}{\sigma}$ โดยที่ μ

คือค่าเฉลี่ย และ σ คือส่วนเบี่ยงเบนมาตรฐาน ภายใต้สมมติฐาน Normal Distribution ประมาณ 99.7% ของข้อมูลควรอยู่ในช่วง $|Z| \leq 3$ ดังนั้นจุดที่มี $|Z| > 3$ จึงถือว่าเป็นผิดปกติ ข้อจำกัดของ Z-score คือไวต่อ Outlier ในการคำนวณ μ และ σ เอง และต้องการข้อมูลที่มีการกระจายแบบ Gaussian

IQR: วิธีการที่ทนทานต่อ Outlier

Interquartile Range (IQR) เป็นวิธีการที่มีความทนทาน (Robust) ต่อ Outlier มากกว่า Z-score เนื่องจากใช้ Median แทน Mean Hastie et al. (2009) อธิบายว่า $IQR = Q_3 - Q_1$ คือช่วงกลาง 50% ของข้อมูล วิธีนี้กำหนดขอบเขต Outlier เป็น $LowerBound = Q_1 - 1.5 \times IQR$ และ $UpperBound = Q_3 + 1.5 \times IQR$ จุดข้อมูลที่อยู่นอกช่วงนี้ถือเป็น Outlier ค่า 1.5 มาจากการสังเกตว่าสำหรับข้อมูล Gaussian จะมีประมาณ 0.7% ของข้อมูลอยู่นอกช่วงนี้ IQR มีข้อดีคือไม่ต้องสมมติการกระจายของข้อมูล และใช้ได้ทั้งกับ Univariate และ Multivariate Data

One-Class SVM: การตรวจจับ Novelty

One-Class SVM เป็นส่วนขยายของ Support Vector Machine สำหรับการตรวจจับ Outlier แบบ Unsupervised Bishop (2006) อธิบายว่า SVM ทำงานโดยการค้นหา $Maximum\ Margin\ Hyperplane$ ซึ่งเป็นระนาบที่แยกข้อมูลสองคลาสออกจากกันด้วยระยะห่างสูงสุด สำหรับ One-Class SVM มีการดัดแปลงเพื่อใช้กับข้อมูลคลาสเดียว โดยหา Hyperplane ที่แยกข้อมูลปกติทั้งหมดออกจาก Origin ด้วย Margin สูงสุด ฟังก์ชันตัดสินใจมีรูปแบบ $f(x) = w^T \phi(x) + b$ โดยที่ $\phi(x)$ คือ Feature Map ที่แปลงข้อมูลไปยัง Feature Space ที่มีมิติสูงกว่า จุดที่อยู่นอก Hyperplane ($f(x) < 0$) ถือเป็น Outlier พารามิเตอร์ ν ควบคุมขอบเขตบนของสัดส่วน Outlier และขอบเขตล่างของ Support Vectors

Isolation Forest: การตรวจจับโดยการแยกแยะ

Isolation Forest เป็นอัลกอริทึมที่ได้รับแรงบันดาลใจจาก Random Forests Hastie et al. (2009) อธิบาย Random Forests ว่าเป็นการรวม (Ensemble) ของต้นไม้การตัดสินใจหลายต้นที่สร้างจาก Bootstrap Sample และใช้ Random Feature Selection ในแต่ละ Split หลักการสำคัญของ Isolation Forest คือ Outlier ถูกแยกได้ง่ายกว่าจุดข้อมูลปกติ เนื่องจาก Outlier มักอยู่ห่างจากข้อมูลส่วนใหญ่ในพื้นที่ Feature ทำให้ต้องการ Split น้อยกว่าเพื่อแยกออกจากข้อมูลอื่น อัลกอริทึมสร้าง Isolation Tree หลายต้น โดยแต่ละต้นสร้างจากการสุ่มเลือก Feature และ Split Value แบบสุ่ม จุดข้อมูลที่ถูกแยกออกได้ด้วยควมลึกเฉลี่ยน้อยกว่า (Anomaly Score สูงกว่า) ถือเป็น Outlier Isolation Forest มีประสิทธิภาพสูง สามารถรองรับข้อมูลมิติสูง และไม่ต้องสมมติการกระจายของข้อมูล

Local Outlier Factor (LOF): การตรวจจับตามความหนาแน่นเฉพาะถิ่น

LOF เป็นวิธีการตรวจจับ Outlier แบบ Density-Based ที่คำนวณความหนาแน่นเฉพาะถิ่นของแต่ละจุดข้อมูลเปรียบเทียบกับเพื่อนบ้าน Hastie et al. (2009) อธิบายว่าการประมาณความหนาแน่นในพื้นที่ (Local Density Estimation) ใช้ค่าเฉลี่ยระยะทางถึงเพื่อนบ้าน k คนแรกเป็นฐาน LOF ของจุด p คำนวณจาก:

$LOF_k(p) = \frac{\sum lrd_k(o) / lrd_k(p)}{|N_k(p)|}$ โดยที่ $lrd_k(p)$ คือ Local Reachability Density

ของจุด p และ $N_k(p)$ คือเซตของเพื่อนบ้าน k คนแรก ค่า $LOF \approx 1$ แสดงว่าจุดนั้นมีความหนาแน่นใกล้เคียงกับเพื่อนบ้าน (ข้อมูลปกติ) ค่า $LOF \gg 1$ แสดงว่าจุดนั้นมีความหนาแน่นต่ำกว่าเพื่อนบ้านมาก (Outlier) ข้อดีของ LOF คือสามารถตรวจจับ Local Outlier ที่อยู่ในบริเวณที่มีความหนาแน่นต่างกันได้ดี ซึ่งวิธีการ Global เช่น Z-score ไม่สามารถทำได้

การประเมินประสิทธิภาพการตรวจจับ Outlier

Tan et al. (2006) เน้นว่าการประเมินประสิทธิภาพของวิธีการตรวจจับ Outlier เป็นงานที่ท้าทาย เนื่องจากชุดข้อมูลมักมี Class Imbalance สูง (Outlier มีน้อยมากเมื่อเทียบกับข้อมูลปกติ) ตัวชี้วัดที่ใช้บ่อยได้แก่ (1) Precision: สัดส่วนของ Outlier ที่ตรวจพบที่เป็น Outlier จริง (2) Recall (Sensitivity): สัดส่วนของ Outlier จริงทั้งหมดที่ถูกตรวจพบ (3) F1-Score: ค่าเฉลี่ยฮาร์โมนิกของ Precision และ Recall (4) AUC-ROC: พื้นที่ใต้เส้นโค้ง ROC ซึ่งวัดความสามารถในการแยกแยะ Outlier จากข้อมูลปกติโดยรวม การเลือกวิธีการตรวจจับ Outlier ที่เหมาะสมขึ้นอยู่กับลักษณะของข้อมูล เป้าหมายของการวิเคราะห์ และต้นทุนของ False Positive (ตรวจพบว่าเป็น Outlier ทั้งที่ไม่ใช่) เทียบกับ False Negative (ตรวจไม่พบ Outlier ที่มีอยู่จริง)

10.2.2 แบบฝึกหัด

10.2.2.1 การสร้างชุดข้อมูล

สร้างชุดข้อมูล 500 จุดปกติตาม Normal Distribution ด้วยค่าเฉลี่ย [5, 10] และส่วนเบี่ยงเบนมาตรฐาน [1, 2] จากนั้นเพิ่ม Outlier 50 จุดด้วยค่าเฉลี่ย [20, 30] และส่วนเบี่ยงเบนมาตรฐาน [5, 8] กำหนดตัวแปรเป้าหมายแบบ Binary (Outlier) เพื่อระบุว่าจุดข้อมูลเป็น Outlier (1) หรือไม่ (0):

```
import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt

np.random.seed(42)
num_data_points = 500
mean_normal = [5, 10]
std_normal = [1, 2]

normal_data_points = np.random.normal(loc=mean_normal,
                                       scale=std_normal, size=(num_data_points, 2))
outliers = np.random.normal(loc=[20, 30],
                             scale=[5, 8], size=(50, 2))

data = np.vstack([normal_data_points, outliers])
target = np.hstack([np.zeros(num_data_points),
                    np.ones(50)])

df = pd.DataFrame(data, columns=['Feature1',
                                'Feature2'])
df['Outlier'] = target.astype(int)

plt.scatter(df['Feature1'], df['Feature2'],
            c=df['Outlier'], cmap='viridis')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Dummy Dataset for Outlier Detection')
plt.colorbar(label='Outlier (1: Yes, 0: No)')
plt.show()
```

ผลลัพธ์: Scatter Plot แสดงจุดข้อมูลปกติและ Outlier แยกสีกัน ชัดเจนว่ากลุ่ม Outlier อยู่ห่างจากกลุ่มข้อมูลหลักมาก

10.2.2.2 Z-score

คำนวณ Z-score ของแต่ละจุดข้อมูลสำหรับ Feature1 และ Feature2 โดยกำหนด Threshold = 3 จุดข้อมูลที่มี Z-score มากกว่า Threshold ใน Feature ใดก็ตามถือเป็น Outlier:

```
from scipy import stats

z_scores = np.abs(stats.zscore(df[['Feature1',
'Feature2']]))
threshold = 3

outliers_zscore = df[(z_scores['Feature1'] >
threshold) |
(z_scores['Feature2'] > threshold)]

plt.scatter(df['Feature1'], df['Feature2'],
c='blue', label='Normal Data Points')
plt.scatter(outliers_zscore['Feature1'],
outliers_zscore['Feature2'],
c='red', label='Outliers (Z-score)')
plt.title('Outlier Detection using Z-score')
plt.legend()
plt.show()
```

หลักการ: Z-score วัดว่าจุดข้อมูลอยู่ห่างจากค่าเฉลี่ยกี่เท่าของส่วนเบี่ยงเบนมาตรฐาน ค่า $|Z| > 3$ แสดงถึงความผิดปกติที่มีนัยสำคัญทางสถิติ เหมาะกับข้อมูลที่มีการกระจายแบบ Normal Distribution

10.2.2.3 IQR

ใช้วิธี Interquartile Range (IQR) ตรวจจับ Outlier โดยจุดข้อมูลที่อยู่นอกช่วง $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ ใน Feature ใดก็ตามถือเป็น Outlier:

```
Q1 = df[['Feature1', 'Feature2']].quantile(0.25)
```

```
Q3 = df[['Feature1', 'Feature2']].quantile(0.75)
IQR = Q3 - Q1

outlier_range_lower = Q1 - 1.5 * IQR
outlier_range_upper = Q3 + 1.5 * IQR

outliers_iqr = df[
    (df['Feature1'] <
outlier_range_lower['Feature1']) |
    (df['Feature1'] >
outlier_range_upper['Feature1']) |
    (df['Feature2'] <
outlier_range_lower['Feature2']) |
    (df['Feature2'] >
outlier_range_upper['Feature2'])
]

plt.scatter(df['Feature1'], df['Feature2'],
            c='blue', label='Normal Data Points')
plt.scatter(outliers_iqr['Feature1'],
            outliers_iqr['Feature2'],
            c='red', label='Outliers (IQR)')
plt.title('Outlier Detection using IQR')
plt.legend()
plt.show()
```

หลักการ: IQR วัดช่วงกลาง 50% ของข้อมูล มีความทนทานต่อ Outlier สูง
เหมาะกับข้อมูลที่ไม่จำเป็นต้องเป็น Normal Distribution

10.2.2.4 One-Class SVM

One-Class SVM เป็น Machine Learning Algorithm สำหรับ Novelty Detection
โดยเรียนรู้ขอบเขตที่แยกข้อมูลส่วนใหญ่ออกจาก Outlier พารามิเตอร์ ν

ควบคุมสัดส่วนของจุดข้อมูลที่ไม่เดลถือว่าเป็น Outlier ผลการทำนาย -1 หมายถึง Outlier และ 1 หมายถึง Inlier:

```
from sklearn.svm import OneClassSVM

X = df[['Feature1', 'Feature2']].values

ocsvm = OneClassSVM(nu=0.15)
ocsvm.fit(X)

outliers_ocsvm = ocsvm.predict(X)
outliers_ocsvm = outliers_ocsvm == -1

plt.scatter(X[:, 0], X[:, 1], c='blue',
            label='Normal Data Points')
plt.scatter(X[outliers_ocsvm, 0],
            X[outliers_ocsvm, 1],
            c='red', label='Outliers (One-Class SVM)')
plt.title('Outlier Detection using One-Class SVM')
plt.legend()
plt.show()
```

หลักการ: One-Class SVM สร้างไฮเปอร์เพลนที่ครอบคลุมข้อมูลปกติให้มากที่สุด จุดที่อยู่นอกขอบเขตถือเป็น Outlier เหมาะกับปัญหา Novelty Detection ที่ข้อมูล Outlier ไม่มีในชุดฝึก

10.2.2.5 Isolation Forest

Isolation Forest เป็นเทคนิคตรวจจับ Outlier แบบ Unsupervised ที่อาศัยแนวคิดการแยกแยะความผิดปกติในข้อมูล โดยสุ่มเลือก Feature และ Split Value

ซ้ำๆ จุดข้อมูลที่ถูกแยกออกได้เร็วกว่า (ต้องการการแยกน้อยกว่า) ถือว่าเป็น Outlier พารามิเตอร์ contamination ควบคุมสัดส่วนที่คาดว่าจะเป็น Outlier:

```
from sklearn.ensemble import IsolationForest

X = df[['Feature1', 'Feature2']].values

isolation_forest = IsolationForest(contamination=0.09)
isolation_forest.fit(X)

outliers_isolation_forest = isolation_forest.predict(X)
outliers_isolation_forest = outliers_isolation_forest == -1

plt.scatter(X[:, 0], X[:, 1], c='blue', label='Normal Data Points')
plt.scatter(X[outliers_isolation_forest, 0],
            X[outliers_isolation_forest, 1],
            c='red', label='Outliers (Isolation Forest)')
plt.title('Outlier Detection using Isolation Forest')
plt.legend()
plt.show()
```

หลักการ: Isolation Forest สร้างต้นไม้การแยกแยกหลายต้น จุดข้อมูลที่ถูกแยกออกได้ด้วยความลึกของต้นไม้ที่น้อยกว่าเฉลี่ยถือเป็น Outlier มีประสิทธิภาพสูงและรองรับข้อมูลมิติสูง

10.2.2.6 DBSCAN

DBSCAN ตรวจจับ Outlier โดยระบุจุดข้อมูลที่ไม่ได้เป็นส่วนหนึ่งของ Cluster ใดๆ พารามิเตอร์ eps กำหนดระยะสูงสุดระหว่างสองตัวอย่างเพื่อให้อยู่ใน Cluster เดียวกัน

และ `min_samples` กำหนดจำนวนตัวอย่างขั้นต่ำใน Neighborhood จุดที่ DBSCAN กำหนด Label เป็น -1 ถือเป็น Outlier:

```
from sklearn.cluster import DBSCAN

X = df[['Feature1', 'Feature2']].values

dbscan = DBSCAN(eps=2.5, min_samples=10)
dbscan.fit(X)

outliers_dbscan = dbscan.labels_ == -1 # DBSCAN
กำหนด -1 ให้ Outlier

plt.scatter(X[:, 0], X[:, 1], c='blue',
            label='Normal Data Points')
plt.scatter(X[outliers_dbscan, 0],
            X[outliers_dbscan, 1],
            c='red', label='Outliers (DBSCAN)')
plt.title('Outlier Detection using DBSCAN')
plt.legend()
plt.show()
```

หลักการ: DBSCAN ระบุ Outlier ว่าเป็นจุดที่ไม่สามารถเป็น Core Point ได้ (มีเพื่อนบ้านน้อยกว่า `min_samples` ในรัศมี `eps`) เหมาะกับข้อมูลที่ Cluster มีรูปทรงซับซ้อน

10.2.2.7 LOF

Local Outlier Factor (LOF) เป็นวิธีการตรวจจับ Outlier ตามความหนาแน่น วัดการเบี่ยงเบนของความหนาแน่นเฉพาะถิ่นของจุดข้อมูลเทียบกับเพื่อนบ้าน จุดที่มีความหนาแน่นเฉพาะถิ่นต่ำกว่าเพื่อนบ้านอย่างมีนัยสำคัญถือเป็น Outlier:

```
from sklearn.neighbors import LocalOutlierFactor

X = df[['Feature1', 'Feature2']].values

lof = LocalOutlierFactor(n_neighbors=100,
contamination=0.10)
outliers_lof = lof.fit_predict(X)
outliers_lof = outliers_lof == -1 # LOF กำหนด -1
ให้ Outlier

plt.scatter(X[:, 0], X[:, 1], c='blue',
label='Normal Data Points')
plt.scatter(X[outliers_lof, 0], X[outliers_lof,
1],
c='red', label='Outliers (LOF)')
plt.title('Outlier Detection using Local Outlier
Factor (LOF)')
plt.legend()
plt.show()
```

หลักการ : L O F
คำนวณอัตราส่วนของความหนาแน่นเฉลี่ยของเพื่อนบ้านต่อความหนาแน่นของจุดนั่นเอง ค่า
LOF >> 1 แสดงว่าจุดนั้นมีความหนาแน่นต่ำกว่าเพื่อนบ้านมาก (Outlier)
เหมาะกับข้อมูลที่มีความหนาแน่นของ Cluster ไม่สม่ำเสมอ

10.2.3 กรณีศึกษา – Credit Card Fraud Detection

ดำเนินการตรวจจับ Outlier บนชุดข้อมูล Credit Card Fraud Detection
โดยครอบคลุมขั้นตอนทั้งหมด 8 ขั้นตอน:

6. โหลดชุดข้อมูล
7. ใช้วิธี Z-score ตรวจจับ Outlier

8. ใช้วิธี IQR ตรวจจับ Outlier
9. ใช้วิธี One-Class SVM ตรวจจับ Outlier
10. ใช้วิธี Isolation Forest ตรวจจับ Outlier
11. ใช้วิธี DBSCAN ตรวจจับ Outlier
12. ใช้วิธี LOF ตรวจจับ Outlier
13. สรุปและเปรียบเทียบผลลัพธ์

10.2.3.1 โหลดชุดข้อมูล

โหลดชุดข้อมูล Credit Card Fraud Detection เข้าสู่ Pandas DataFrame สุ่มตัวอย่าง 10,000 รายการ แล้วสำรวจข้อมูลเบื้องต้น:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import OneClassSVM
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.cluster import DBSCAN

df = pd.read_csv('/content/creditcard.csv')
df = df.sample(10000)

print(df.head())
print(df.info())
print(df.describe())
df['Class'].value_counts()
```

ผลลัพธ์: DataFrame 10,000 แถว $\times$ 31 คอลัมน์ ประกอบด้วย Time, V1–V28 (Feature ที่ผ่าน PCA), Amount และ Class (0 = ปกติ, 1 = ทูจริต) พบการทูจริตเพียง 16 รายการ (0.16%) จาก 10,000 รายการ แสดงถึงปัญหา Class Imbalance ที่รุนแรง

10.2.3.2 ใช้วิธี Z-score

คำนวณ Z-score สำหรับทุก Feature (ยกเว้น Class) โดยกำหนด Threshold = 3 จุดข้อมูลที่มี Z-score เกิน Threshold ใน Feature ใดก็ตามถือเป็น Outlier:

```
z_scores = np.abs(StandardScaler().fit_transform(df.drop('Class', axis=1)))
threshold = 3
outliers_zscore = (z_scores > threshold).any(axis=1)

plt.bar(['Normal', 'Outlier'],
        [len(df) - outliers_zscore.sum(), outliers_zscore.sum()])
plt.xlabel(f'Data Points {len(df) - outliers_zscore.sum()}: {outliers_zscore.sum()}')
plt.ylabel('Count')
plt.title('Outliers Detected by Z-score Method')
plt.show()
df[outliers_zscore]['Class'].value_counts()
```

ผลลัพธ์: Z-score ตรวจพบ Outlier จำนวน 1,361 รายการ ใน 1,361 รายการนั้น เป็นการทูจริตจริง 13 รายการ และ Normal 1,348 รายการ แสดงว่าวิธีนี้ตรวจพบ False Positive จำนวนมาก

10.2.3.3 ใช้วิธี IQR

ใช้วิธี IQR กับทุก Feature (ยกเว้น Class) โดยจุดข้อมูลที่อยู่นอกช่วง $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ ใน Feature ใดก็ตามถือเป็น Outlier:

```
Q1 = df.drop('Class', axis=1).quantile(0.25)
Q3 = df.drop('Class', axis=1).quantile(0.75)
IQR = Q3 - Q1

outlier_range_lower = Q1 - 1.5 * IQR
outlier_range_upper = Q3 + 1.5 * IQR

outliers_iqr = ((df.drop('Class', axis=1) <
outlier_range_lower) |
(df.drop('Class', axis=1) >
outlier_range_upper)).any(axis=1)

plt.bar(['Normal', 'Outlier'],
[ len(df) - outliers_iqr.sum(),
outliers_iqr.sum() ])
plt.title('Outliers Detected by IQR Method')
plt.show()
df[outliers_iqr]['Class'].value_counts()
```

ผลลัพธ์: IQR ตรวจพบ Outlier จำนวน 4,777 รายการ ใน 4,777 รายการนั้น เป็นการทุจริตจริง 16 รายการ (ตรวจพบครบทุกราย) และ Normal 4,761 รายการ วิธีนี้ Recall สูง แต่มี False Positive สูงมากเช่นกัน

10.2.3.4 ใช้วิธี One-Class SVM

ใช้ One-Class SVM ตรวจจับ Outlier โดยกำหนด $\nu = 0.1$ (คาดว่า 10% เป็น Outlier):

```
df_svm = df.dropna()
ocsvm = OneClassSVM(nu=0.1)
outliers_ocsvm =
ocsvm.fit_predict(df_svm.drop('Class', axis=1))
```

```

outliers_ocsvm = outliers_ocsvm == -1

plt.bar(['Normal', 'Outlier'],
        [len(df_svm) - outliers_ocsvm.sum(),
         outliers_ocsvm.sum()])
plt.title('Outliers Detected by One-Class SVM
Method')
plt.show()
df[outliers_ocsvm]['Class'].value_counts()

```

ผลลัพธ์: One-Class SVM ตรวจพบ Outlier 999 รายการ ใน 999 รายการนั้น เป็นการทุจริตจริง 1 รายการ และ Normal 998 รายการ วิธีนี้ตรวจพบ Fraud น้อยมากบนชุดข้อมูลนี้

10.2.3.5 ใช้วิธี Isolation Forest

ใช้ Isolation Forest ตรวจจับ Outlier โดยกำหนด contamination = 0.02 (คาดว่า 2% เป็น Outlier):

```

isolation_forest =
IsolationForest(contamination=0.02)
outliers_isolation_forest =
isolation_forest.fit_predict(df.drop('Class',
axis=1))
outliers_isolation_forest =
outliers_isolation_forest == -1

plt.bar(['Normal', 'Outlier'],
        [len(df) - outliers_isolation_forest.sum(),
         outliers_isolation_forest.sum()])
plt.title('Outliers Detected by Isolation Forest
Method')
plt.show()
df[outliers_isolation_forest]['Class'].value_co
unts()

```

ผลลัพธ์: Isolation Forest ตรวจพบ Outlier 200 รายการ ใน 200 รายการนั้น เป็นการทุจริตจริง 10 รายการ และ Normal 190 รายการ ทำได้ดีที่สุดในแง่สัดส่วน Fraud ที่ตรวจพบ ($10/16 = 62.5\%$)

10.2.3.6 ใช้วิธี DBSCAN

ใช้ DBSCAN ตรวจจับ Outlier โดยกำหนด $\text{eps} = 100$ และ $\text{min_samples} = 5$ จุดที่ถูก Label เป็น -1 ถือเป็น Outlier:

```
dbscan = DBSCAN(eps=100, min_samples=5)
outliers_dbscan = dbscan.fit_predict(df.drop('Class', axis=1))
outliers_dbscan = outliers_dbscan == -1

plt.bar(['Normal', 'Outlier'],
        [len(df) - outliers_dbscan.sum(),
         outliers_dbscan.sum()])
plt.title('Outliers Detected by DBSCAN Method')
plt.show()
df[outliers_dbscan]['Class'].value_counts()
```

ผลลัพธ์: DBSCAN ตรวจพบ Outlier 1,480 รายการ ใน 1,480 รายการนั้น เป็นการทุจริตจริง 8 รายการ และ Normal 1,472 รายการ

10.2.3.7 ใช้วิธี LOF

ใช้ LOF ตรวจจับ Outlier โดยกำหนด $n_neighbors = 50$ และ $\text{contamination} = 0.02$:

```
lof = LocalOutlierFactor(n_neighbors=50,
                          contamination=0.02)
outliers_lof = lof.fit_predict(df.drop('Class',
                                       axis=1))
outliers_lof = outliers_lof == -1
```

```
plt.bar(['Normal', 'Outlier'],
        [len(df) - outliers_lof.sum(),
         outliers_lof.sum()])
plt.title('Outliers Detected by Local Outlier
Factor (LOF) Method')
plt.show()
df[outliers_lof]['Class'].value_counts()
```

ผลลัพธ์: LOF ตรวจพบ Outlier 200 รายการ ใน 200 รายการนั้น เป็นการทุจริตจริง 2 รายการ และ Normal 198 รายการ

10.2.3.8 สรุป

ในบริบทเฉพาะนี้ ไม่ใช่ทุกโมเดลที่ทำงานได้ดี! สรุปผลการเปรียบเทียบวิธีการตรวจจับ Outlier ทั้งหมดบนชุดข้อมูล Credit Card Fraud Detection:

14. IQR: ตรวจพบ Fraud ได้ครบ 16/16 ราย แต่มี False Positive สูงมาก (4,761 ราย)
15. Z-score: ตรวจพบ Fraud ได้ 13/16 ราย มี False Positive 1,348 ราย
16. DBSCAN: ตรวจพบ Fraud ได้ 8/16 ราย มี False Positive 1,472 ราย
17. Isolation Forest: ตรวจพบ Fraud ได้ 10/16 ราย มี False Positive เพียง 190 ราย — ดีที่สุดในแง่สมดุลระหว่าง Recall และ Precision
18. LOF: ตรวจพบ Fraud ได้เพียง 2/16 ราย มี False Positive 198 ราย
19. One-Class SVM: ตรวจพบ Fraud ได้เพียง 1/16 ราย — ทำงานได้ไม่ดีบนชุดข้อมูลนี้

บทเรียนที่ได้: การเลือกวิธีการตรวจจับ Outlier ที่เหมาะสมขึ้นอยู่กับลักษณะของข้อมูลและเป้าหมายของการวิเคราะห์ Isolation Forest เหมาะสมที่สุดสำหรับการตรวจจับ Fraud บนชุดข้อมูลนี้

เนื่องจากให้สมดุลที่ดีระหว่างการตรวจพบ Fraud จริงและการลด False Positive สามารถปรับ Hyperparameter ของแต่ละวิธีเพื่อเพิ่มประสิทธิภาพตามกรณีการใช้งานเฉพาะ

ตัวอย่างการประยุกต์ใช้งานจากงานวิจัย

การฉ้อโกงประกันภัย (Insurance Fraud) เป็นตัวอย่างของ Global Outlier ในข้อมูลจริง กล่าวคือการเรียกร้องค่าสินไหมปลอมมีลักษณะแตกต่างจาก การเรียกร้องปกติอย่างชัดเจน สิมมาจันท์และคณะ (2023) ในงานวิจัยเรื่อง "Detecting Fraudulent Claims in Automobile Insurance Policies by Data Mining Techniques" ได้ใช้เทคนิคเหมืองข้อมูลเพื่อตรวจจับการเรียกร้องที่ผิดปกติ จากชุดข้อมูลที่ไม่สมดุล (Imbalanced Data) ซึ่งสัดส่วนของการฉ้อโกง มีน้อยมากเมื่อเทียบกับการเรียกร้องปกติ ลักษณะนี้สอดคล้องกับแนวคิด ของ Outlier Detection ที่การสังเกตส่วนน้อย (Minority Class) คือค่าที่ผิดปกติซึ่งต้องการตรวจจับ

ผาพันธ์และคณะ (2025) ได้พัฒนาต่อยอดในงานวิจัยเรื่อง "Detecting Automobile Insurance Fraud: A Novel Two-Step Strategy Using Effective Ensemble Learning Techniques" โดยนำ SMOTE มาช่วยสร้างตัวอย่างสังเคราะห์ของ Outlier (Minority Class) เพื่อให้แบบจำลองสามารถเรียนรู้ลักษณะเฉพาะของค่าผิดปกติ ได้ดียิ่งขึ้น และใช้ Mutual Information ในการระบุตัวแปรที่ช่วยแยกแยะ Outlier ออกจากข้อมูลปกติ ผลการวิจัยแสดงให้เห็นว่าการประยุกต์ใช้ Ensemble Learning ช่วยเพิ่มประสิทธิภาพในการตรวจจับค่าผิดปกติใน ข้อมูลที่ไม่สมดุลได้อย่างมีนัยสำคัญ

เอกสารอ้างอิง

1. Simmachan, T., Manopa, W., Neamhom, P., Poothong, A., & Phaphan, W. (2023). Detecting fraudulent claims in automobile insurance policies by data mining techniques. *Thailand Statistician*, 21(3), 552–568.

2. Phaphan, W., Sathitvudh, S., Suntornsuwan, T., Budsaba, K., & Simmachan, T. (2025). Detecting automobile insurance fraud: A novel two-step strategy using effective ensemble learning techniques. *Thailand Statistician*, 23(1), 144–161.